Universidad de Buenos Aires

Facultad de Ingeniería

Redes Neuronales (86.54)

Trabajo Practico N°3

2°C 2024

Alumno: Milidoni, Cristian Julian

Padrón: 107647

Diciembre 2024

Contents

1 Ejercicio 8 - TP2 **2**
1.1 Enunciado 2
1.2 Simulated Annealing 2
1.3 Implementación y resultados 2

2 Ejercicio 1 - TP3 **5**
2.1 Enunciado 5
2.2 Red de Kohonen 5
2.3 Implementación y resultados 6

3 Ejercicio 2 - TP3 **9**
3.1 Enunciado 9
3.2 Traveling Salesman Problem 9
3.3 Implementación y resultados 10

1 Ejercicio 8 - TP2

1.1 Enunciado

Encontrar un perceptrón multicapa que resuelva una XOR de 2 entradas mediante simulated annealing. Graficar el error a lo largo del proceso de aprendizaje.

1.2 Simulated Annealing

El *Simulated Annealing* (SA) es un algoritmo de optimización estocástico que emula el proceso físico de enfriamiento de un material, donde á altas temperaturas, los *átomos* se mueven libremente, pero al reducir la temperatura, alcanzan un estado de mínima energía. El algoritmo busca soluciones *óptimas* iterativamente, permitiendo transiciones hacia estados peores con una probabilidad que disminuye conforme la temperatura baja.

Los pasos del algoritmo son los siguientes:

1. **Inicialización**: Se genera una solución aleatoria inicial y se establece una temperatura inicial alta.
2. **Perturbación**: Se modifica ligeramente la solución actual, ajustando los pesos y sesgos del modelo de manera aleatoria dentro de un rango pequeño.
3. **Evaluación**: Se calcula la calidad de la nueva solución mediante una función de error, como el error cuadrático medio (MSE).
4. **Aceptación o Rechazo**: Si la nueva solución es mejor (error menor), se acepta. Si es peor, se acepta con una probabilidad decreciente con la temperatura, basada en la función $e^{-\Delta E/T}$, donde ΔE es la diferencia de error y T es la temperatura.
5. **Enfriamiento**: Se reduce la temperatura multiplicándola por un factor menor que 1 (*enfriamiento exponencial*).
6. **Iteración**: Los pasos anteriores se repiten hasta alcanzar un número máximo de iteraciones.

1.3 Implementación y resultados

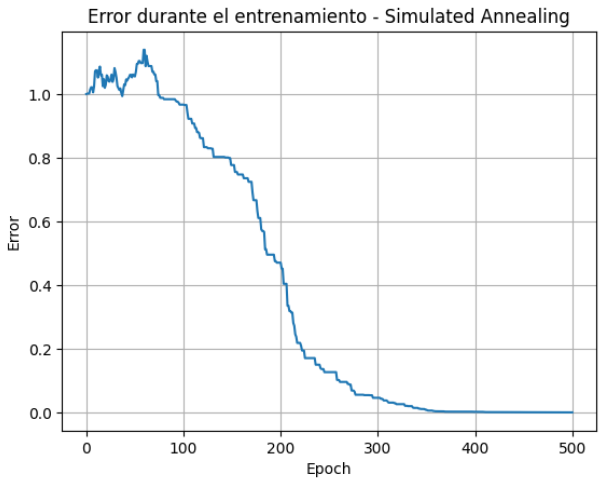
A la implementación del perceptrón multicapa de anteriores ejercicios, se le adicionó un método para entrenarse por medio de *Simulated Annealing*.

```
def simulated_annealing(self, X, y, epochs, initial_temp, cooling_rate):
    best_weights = [layer.weights.copy() for layer in self.layers]
    best_biases = [layer.biases.copy() for layer in self.layers]
    best_output = self.forward(X)
    best_error = np.mean(np.square(best_output - y))
    errors = [best_error]
    temperature = initial_temp

    for _ in range(epochs):
        for layer in self.layers:
            layer.weights += np.random.uniform(-0.1, 0.1, layer.weights.shape)
            layer.biases += np.random.uniform(-0.1, 0.1, layer.biases.shape)

        current_output = self.forward(X)
        current_error = np.mean(np.square(current_output - y))
        delta_error = current_error - best_error
        if delta_error < 0 or random.random() < np.exp(-delta_error / temperature):
            best_weights = [layer.weights.copy() for layer in self.layers]
            best_biases = [layer.biases.copy() for layer in self.layers]
            best_error = current_error
        else:
            for layer, bw, bb in zip(self.layers, best_weights, best_biases):
                layer.weights = bw.copy()
                layer.biases = bb.copy()

        temperature *= cooling_rate
        errors.append(best_error)

    self.errors = errors
```

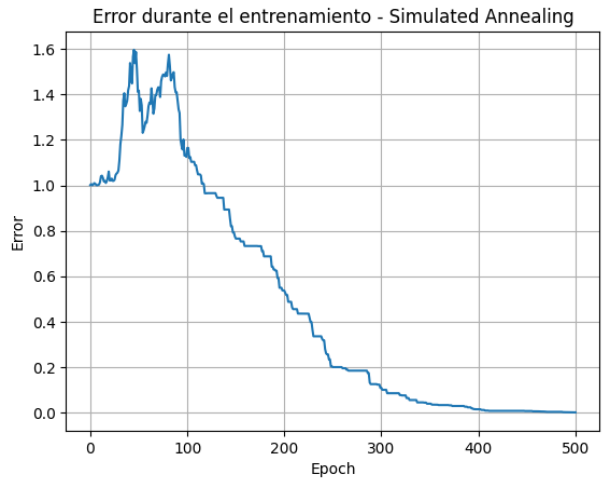
El rendimiento del algoritmo de Simulated Annealing depende de los siguientes parámetros principales:

- **Temperatura inicial** (T_i): Determina que tan libremente puede explorar el algoritmo en las etapas iniciales.
- **Tasa de enfriamiento** (α): Define la rapidez con la que disminuye la temperatura en cada iteración.
- **Cantidad de épocas** (N_{epocas}): Controla el número de iteraciones que realiza el algoritmo para optimizar la solución.

Para empezar, se entrenó a un perceptrón multicapa de 4 neuronas en su capa oculta y función de activación *tanh()* por medio de simulated annealing por 500 épocas con una temperatura inicial $T_i = 100°C$ y una tasa de enfriamiento $\alpha = 0.90$, es decir, reduciendo la temperatura un 10% en cada época. Luego, se entrenó a otro perceptrón multicapa de la mismas características durante la misma cantidad de epocas pero con $T_i = 300°C$ y $\alpha = 0.85$. En la Fig. 6 se observan los resultados de ambas experiencias.



(a) Entrenamiento con $T_i = 300°C$ y $\alpha = 0.85$



(b) Entrenamiento con $T_i = 100°C$ y $\alpha = 0.92$

Figure 1: Error durante el entrenamiento para ambas experiencias

Resultado Esperado	Resultado Obtenido
-1	-0.99059304
1	0.98832283
1	0.99647911
-1	-0.99047179

Table 1: Entrenamiento con $T_i = 300°C$ y $\alpha = 0.85$

Resultado Deseado	Resultado Obtenido
-1	-0.97817536
1	0.96459717
1	0.97939852
-1	-0.96410495

Table 2: Entrenamiento con $T_i = 100°C$ y $\alpha = 0.92$

Notar que se pueden obtener resultados similares combinando una temperatura inicial baja con una tasa de enfriamiento alta o una temperatura inicial alta con una tasa de enfriamiento baja.

Una de las principales ventajas del *Simulated Annealing* es su capacidad para evitar quedar atrapado en mínimos locales del error durante el entrenamiento. Una temperatura inicial alta permite explorar un espacio de soluciones más amplio en las primeras iteraciones, aumentando la probabilidad de escapar de mínimos locales. En contraste, una temperatura inicial baja puede llevar a soluciones rápidas, pero con un mayor riesgo de quedar atrapado en mínimos locales debido a la exploración limitada del espacio de soluciones.

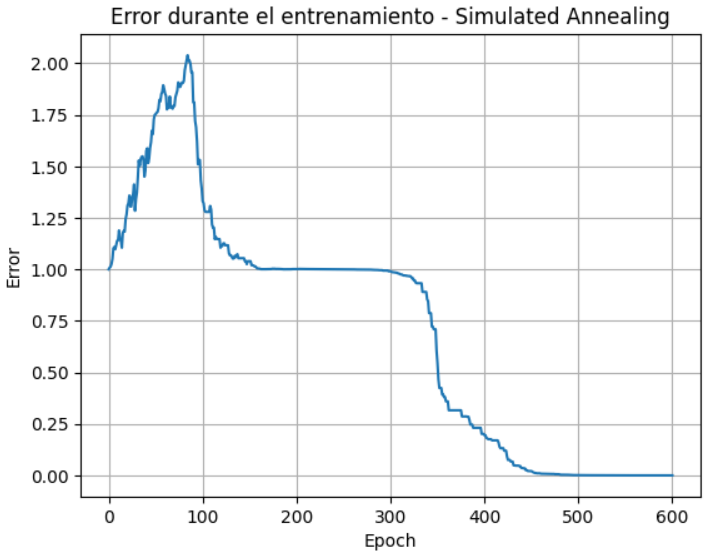


Figure 2: Capacidad de escapar de mínimos locales del algoritmo *Simulated Annealing*

Además, un factor de enfriamiento más lento o un mayor número de épocas durante el entrenamiento aumenta la probabilidad de alcanzar una solución cercana al óptimo global. Esto se debe a que el algoritmo tiene más tiempo para explorar y ajustar antes de "congelarse" en un estado subóptimo o finalizar el proceso de entrenamiento.

Por último, una ventaja adicional del *Simulated Annealing* es que no requiere derivadas, lo que permite su aplicación en casos donde las funciones de activación no son derivables. Esto amplía su rango de aplicaciones y facilita su implementación en diversos contextos.

2 Ejercicio 1 - TP3

2.1 Enunciado

Construya una red de Kohonen de 2 entradas que aprenda una distribución uniforme dentro del círculo unitario. Mostrar el mapa de preservación de topología. Probar con distribuciones uniformes dentro de otras figuras geométricas.

2.2 Red de Kohonen

Las *redes de Kohonen*, también conocidas como mapas auto-organizados (Self-Organizing Maps, SOM), son un tipo de red neuronal no supervisada que aprende a representar patrones de entrada a través de un proceso iterativo, donde las neuronas se ajustan de acuerdo con las características de los datos, preservando la topología de los mismos.

La red de Kohonen consiste en una cuadrícula de neuronas organizadas topológicamente. Cada neurona está asociada a un vector de pesos $\mathbf{w}_i \in \mathbb{R}^d$, donde d es la dimensión de los datos de entrada. El aprendizaje se basa en la adaptación iterativa de los pesos para acercarse a los datos, preservando la estructura topológica de la cuadrícula.

El funcionamiento de la red de Kohonen se divide en los siguientes pasos principales:

1. **Inicialización:** Los pesos de las neuronas se inicializan aleatoriamente con valores pequeños.
2. **Competencia:** Para cada dato de entrada $\mathbf{x}$, se determina la neurona ganadora i^*, aquella cuya distancia a $\mathbf{x}$ es mínima:
$$i^* = \arg\min_i \|\mathbf{x} - \mathbf{w}_i\|$$
3. **Cooperación:** Las neuronas cercanas a la ganadora (en términos de la topología de la cuadrícula) se actualizan utilizando una función de vecindad $V(i, i^*)$:
$$V(i, i^*) = \exp\left(-\frac{\text{dist}(i, i^*)^2}{2\sigma^2}\right)$$
donde $\text{dist}(i, i^*)$ es la distancia topológica entre las neuronas i e i^* y σ es el ancho de la vecindad.
4. **Adaptación:** Los pesos de las neuronas se ajustan con una tasa de aprendizaje η:
$$\Delta\mathbf{w}_i = \eta \cdot V(i, i^*) \cdot (\mathbf{x} - \mathbf{w}_i).$$

El modelo se entrena iterativamente sobre los datos de entrada, actualizando los pesos y reduciendo σ en cada época para limitar gradualmente la vecindad de actualización.

Es muy útil que la red implementada pueda utilizar una cuadrícula de neuronas de dimensiones especificas según la aplicación, esto se va demostrar con la diferencia de dimensiones que hay entre la cuadricula de neuronas del **Ejercicio 1** y del **Ejercicio 2**.

2.3 Implementación y resultados

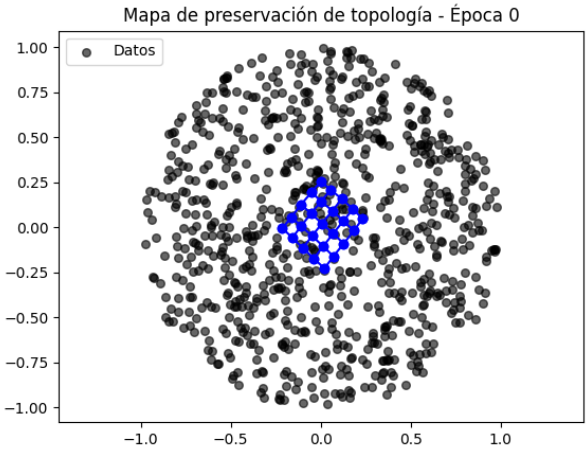
Se implemento la Red de Kohonen de la siguiente forma:

```
class Kohonen:
    def __init__(self, dim, N, filas, columnas):
        self.dim = dim
        self.N = N
        self.filas = filas
        self.columnas = columnas
        self.w = np.random.normal(0, 0.1, (dim, N))
        self.dist_matrix = self.calcular_distancia_neuronas()

    def calcular_distancia_neuronas(self):
        dist_matrix = np.zeros((self.N, self.N))
        for i in range(self.filas):
            for j in range(self.columnas):
                idx1 = i * self.columnas + j
                for k in range(self.filas):
                    for l in range(self.columnas):
                        idx2 = k * self.columnas + l
                        dist_matrix[idx1, idx2] = np.sqrt((i - k)**2 + (j - l)**2)
        return dist_matrix

    def train(self, X, sigma, eta, epochs):
        N_x = X.shape[0]
        min_dists = np.zeros(N_x * epochs)
        sigma_e = sigma
        for e in range(epochs):
            np.random.shuffle(X)
            for i, x in enumerate(X):

                distancias = np.linalg.norm(self.w - x[:, np.newaxis], axis=0)
                winner = np.argmin(distancias)
                min_dists[i + e * N_x] = distancias[winner]

                V = np.exp(-self.dist_matrix[:, winner]**2 / (2 * sigma_e**2))

                delta_w = eta * V * (-self.w.T + x).T
                self.w += delta_w

            sigma_e = sigma_e * 0.99

        return min_dists
```

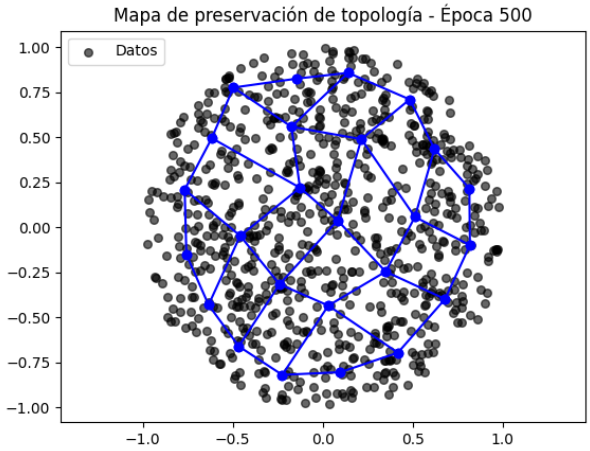
Observar que la clase *Kohonen* recibe como parámetro como se van a organizar las neuronas en términos de la topología de la cuadricula y, como la distancia entre cada par de neuronas es constante, el método *calcular_distancias_neuronas()* realiza un precálculo de esta distancia para ahorrar tiempo de cómputo durante el entrenamiento.

Para entrenar la red de Kohonen, se generaron 700 puntos distribuidos uniformemente dentro del círculo unitario para ser utilizados como datos de entrada en el proceso de aprendizaje. La red utilizada tenia una tasa de aprendizaje $\eta = 0.01$, un ancho de vecindad $\sigma = 3$, un factor de reducción de σ del 1% y 25 neuronas que se organizaron en un arreglo de dimensiones 5x5. La red fue entrenada durante 1000 épocas, actualizando los pesos de las neuronas en cada iteración según el algoritmo de Kohonen.

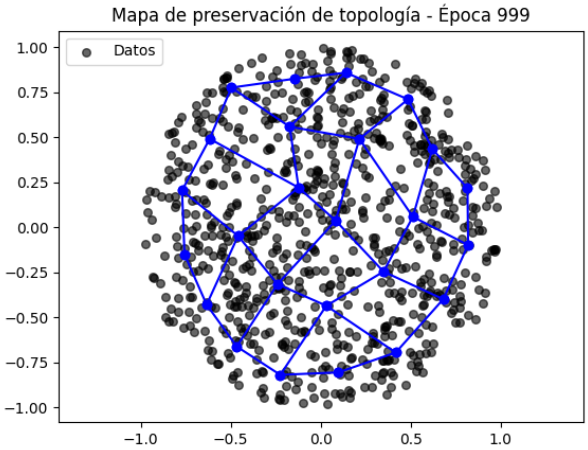
En la Fig. 3 se muestra la evolución de la red durante el entrenamiento.



(a) Evolución del entrenamiento - inicio



(b) Evolución del entrenamiento - mitad

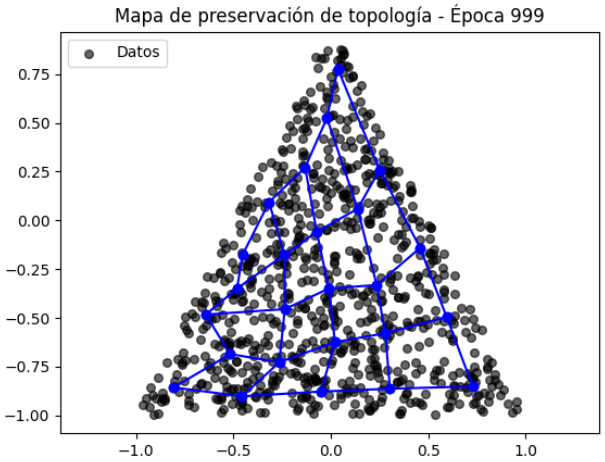


(c) Evolución del entrenamiento - final

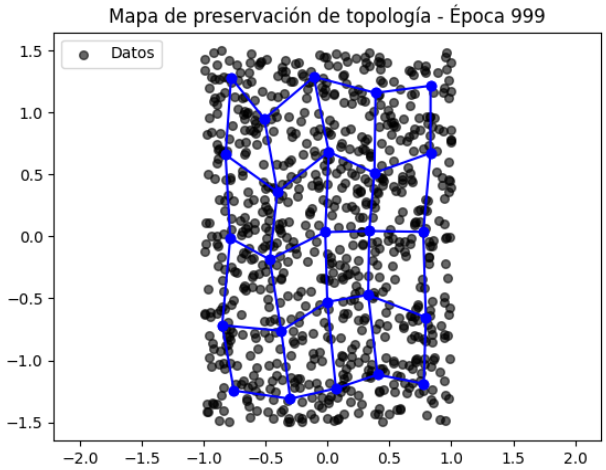
Figure 3: Evolución del entrenamiento de la red de Kohonen con distribución circular uniforme de datos

Los resultados obtenidos muestran cómo la red adaptó sus pesos para representar adecuadamente la distribución de los puntos, preservando la topología del espacio circular de entrada. En el proceso, las neuronas cercanas a la neurona ganadora se ajustaron de acuerdo con la vecindad, y el valor de σ disminuyó gradualmente para enfocar el aprendizaje en las vecindades más cercanas conforme avanzaban las épocas.

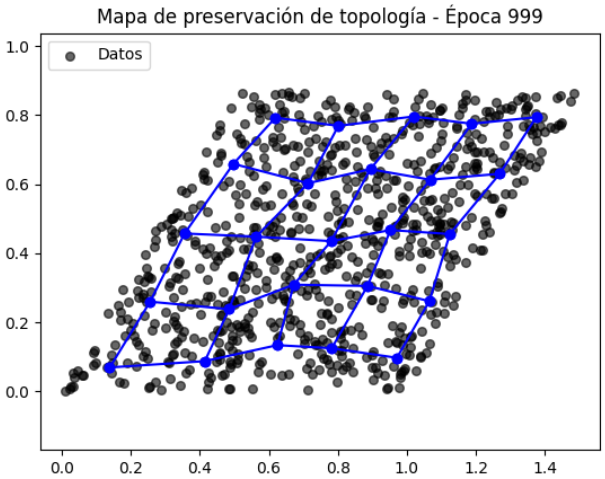
Además, manteniendo los mismos parámetros de la red de anterior, se probo entrenar la red de Kohonen con distintas distribuciones uniformes de datos dentro de otras figuras geométricas. En todos los casos la red logró adaptar sus pesos sinápticos para representar cada distribución de los datos de entrenamiento como se muestra en la Fig. 4.



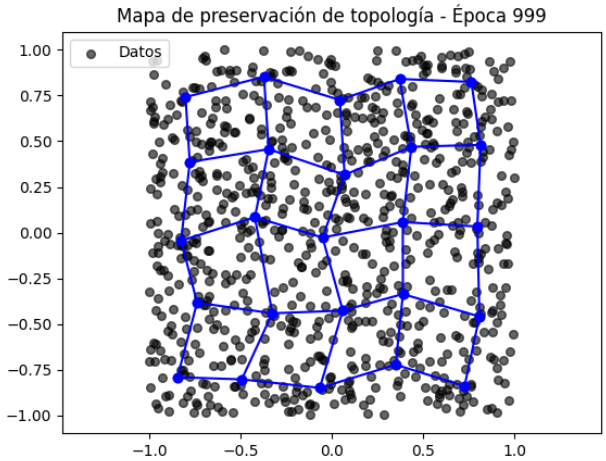
(a) Resultado final - Distribución Triangular



(b) Resultado final - Distribución Rectangular



(c) Resultado final - Distribución Paralelogramica



(d) Resultado final - Distribución Cuadrada

Figure 4: Resultados finales de la red entrenada con distintas distribuciones geométricas uniformes de datos

3 Ejercicio 2 - TP3

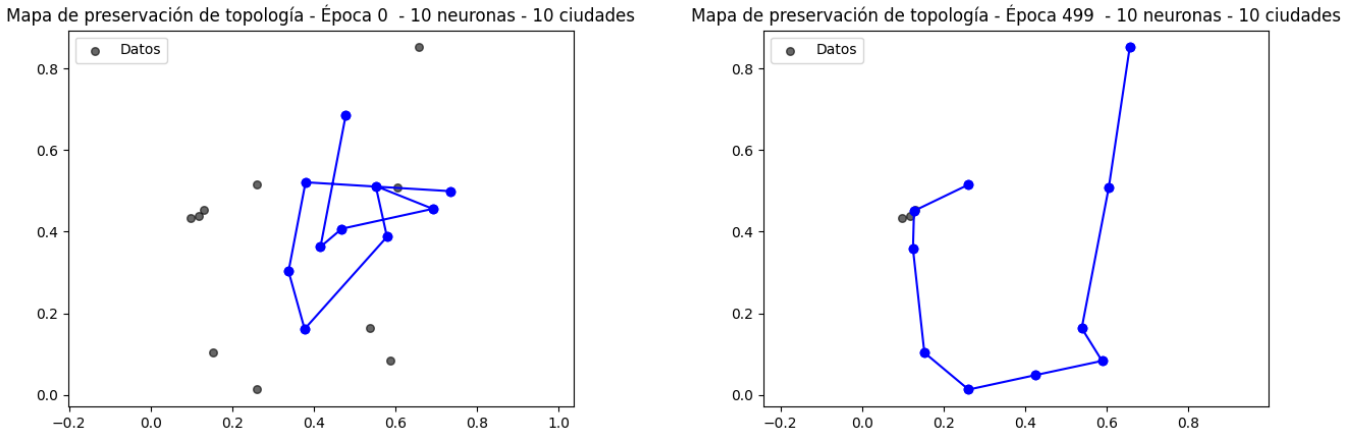
3.1 Enunciado

Resuelva (aproximadamente) el “Traveling salesman problem” para 200 ciudades con una red de Kohonen.

3.2 Traveling Salesman Problem

El ”Traveling Salesman Problem” (TSP, por sus siglas en inglés) consiste en encontrar la ruta más corta que permite a un vendedor itinerante visitar un conjunto de ciudades exactamente una vez, regresando finalmente a la ciudad de origen.

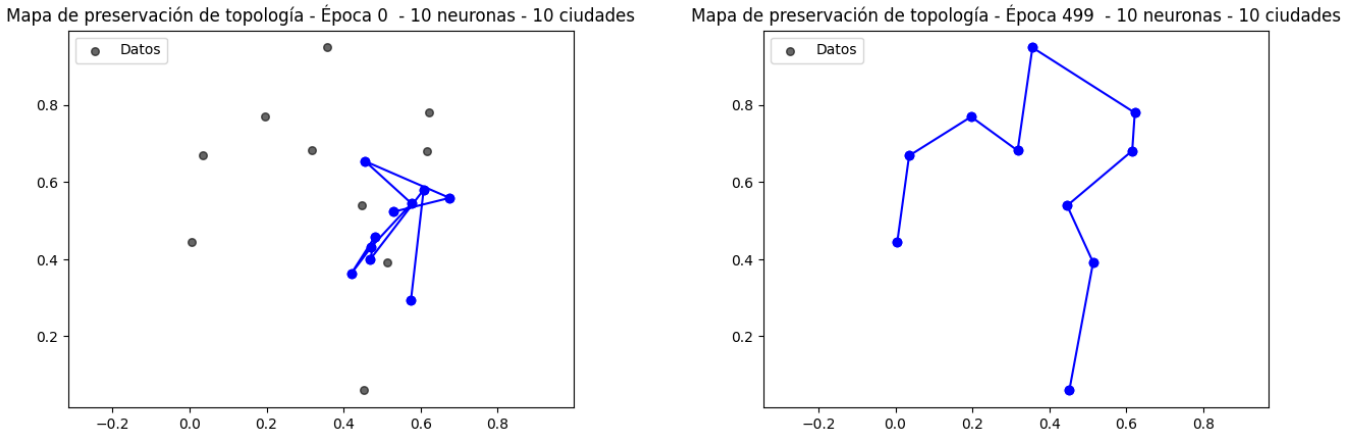
Se presenta una solución del TSP utilizando redes de Kohonen, también conocidas como mapas auto-organizados (Self-Organizing Maps, SOM). Estas redes neuronales no supervisadas proyectan datos en un espacio de menor dimensión, preservando relaciones topológicas, lo que permite aproximar soluciones al problema. Se pueden encontrar tanto soluciones optimas como subóptimas. Las soluciones optimas se alcanzan cuando las neuronas de la red logran posicionarse encima de los puntos que representan ciudades.



(a) TSP de 10 ciudades - inicio

(b) TSP de 10 ciudades - final

Figure 5: Solución no optima de TSP de 10 ciudades con red de Kohonen de 10 neuronas



(a) TSP de 10 ciudades - inicio

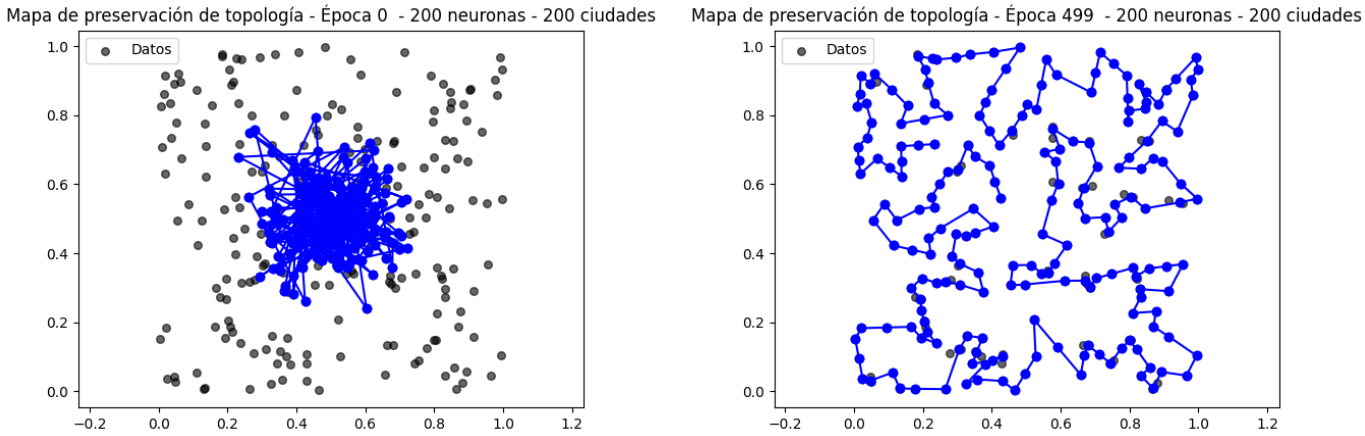
(b) TSP de 10 ciudades - final

Figure 6: Solución óptima de TSP de 10 ciudades con red de Kohonen de 10 neuronas

3.3 Implementación y resultados

Se utilizo la misma red de Kohonen del ejercicio anterior solo que las neuronas se organizan en un arreglo unidimensional, una al lado de la otra.

Se intento resolver el TSP de 200 ciudades utilizando una red de Kohonen de 200 neuronas. La red de Kohonen utilizada tenia $\sigma = 25$ y $\eta = 0.8$ y se entrenó durante 500 épocas. Los resultados encontrados se muestran en la Fig. 7.



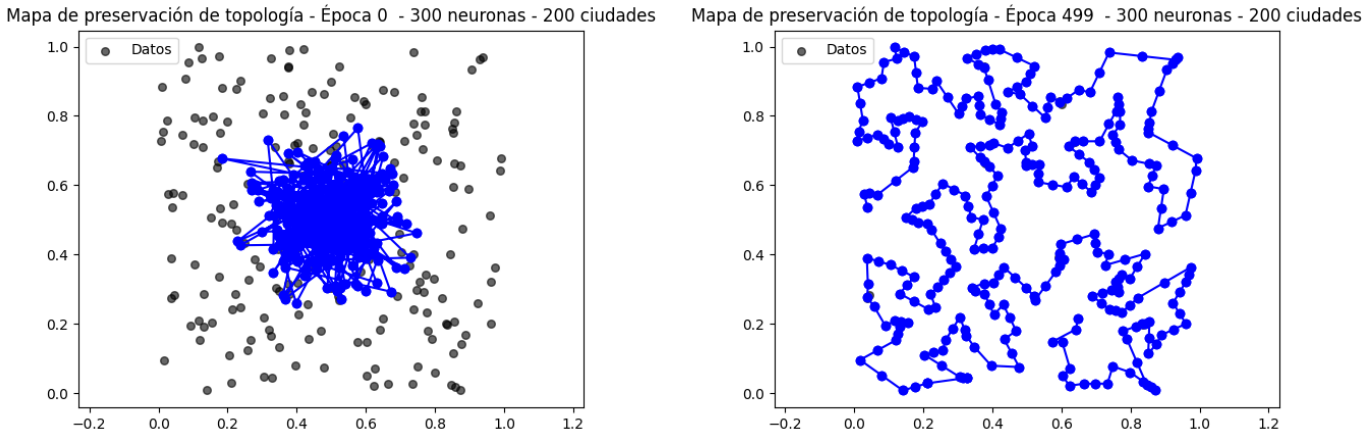
(a) TSP de 10 ciudades - inicio

(b) TSP de 10 ciudades - final

Figure 7: Solución óptima de TSP de 10 ciudades con red de Kohonen de 10 neuronas

Notar que la solución hallada es subóptima. Se intento encontrar una solución optima cambiando el ancho de la vencidad σ, la tasa de aprendizaje η o la cantidad de épocas del entrenamiento pero en ningún caso se consiguió que las neuronas se posicionaran encima de las ciudades.

Se puede conseguir una solución mejor donde no quede ninguna ciudad sin recorrer creando una red de Kohonen con mas neuronas que ciudades. En la fig. 8 se observa una solución encontrada al TSP de 200 ciudades usando una red de Kohonen de 300 neuronas.



(a) TSP de 10 ciudades - inicio

(b) TSP de 10 ciudades - final

Figure 8: Solución óptima de TSP de 10 ciudades con red de Kohonen de 10 neuronas

Si bien no es una solución óptima ya que el viajero no pasaría solo por las ciudades del problema, al menos con esta solución se asegura de pasar por todas las ciudades y se obtiene una aproximación del camino mas corto posible.